Mock C - Marking Scheme and Walkthrough

Practical-assessment style: pandas, matplotlib, SQLite, PyTorch. 40 marks. Model solution: `MockC_solution.py` *(tested twice from a clean copy: Python 3.14, pandas 3.1, matplotlib 3.11, torch 2.14). Data:* `customers.csv` *(83 data rows) and* `orders.csv` *(134 data rows).*

Remember the instructor's advice: do not memorise one complete solution. Every row of the tables below is ONE small operation with its own mark. Mark yourself operation by operation, and revise the single operations you missed.

1. Checklist bullet -> where marked -> topic document to revise

Instructor checklist bullet	Marked in	Revise
Pandas: loading CSV files	1a (2)	`T5_Pandas_NumPy` (and `T3_Files_CSV_Exceptions` for plain CSV files)
checking the shape of a DataFrame	1b (1), also 1c/1d printouts	`T5_Pandas_NumPy`
identifying and removing duplicates	1c (3)	`T5_Pandas_NumPy`
merging DataFrames using a common column	1d (3)	`T5_Pandas_NumPy` (merge / join part)
selecting and filtering data	1e (2)	`T5_Pandas_NumPy`
saving a DataFrame to a CSV file	1f (1), 3d (part)	`T5_Pandas_NumPy`, `T3_Files_CSV_Exceptions`
Matplotlib: scatter, line, bar, box plots	2a-2d (4 x 1)	`T6_Matplotlib` (incl. box plot part)
titles and axis labels	2a-2d (4 x 1), 4f	`T6_Matplotlib`
SQLite: creating and connecting to a database	3a (1)	`T4_SQLite`
storing a DataFrame in a table	3a (2)	`T4_SQLite` and `T5_Pandas_NumPy` (to_sql)
SELECT, WHERE, ORDER BY	3b (1), 3d (1)	`T4_SQLite`
UPDATE queries	3c (1)	`T4_SQLite` (UPDATE part)
parameterised queries with ?	3b (1), 3c	`T4_SQLite`
committing database changes	3c (1)	`T4_SQLite`
retrieving results and exporting them to CSV	3d (2)	`T4_SQLite` (export part), `T3_Files_CSV_Exceptions`
PyTorch: random seed before generating data	4a (1)	`T7_PyTorch_Basics`
defining a network with nn.Module, Linear + ReLU	4c (2)	`T7_PyTorch_Basics`
synthetic data y = X @ w + b + noise	4a (2)	`T7_PyTorch_Basics` (regression part)
understanding the shapes of X and y	4b (1), and `(200,1)` choice in 4a	`T7_PyTorch_Basics`
choosing the loss function and optimiser	4d (1)	`T7_PyTorch_Basics`
a basic training loop	4e (1)	`T7_PyTorch_Basics`
recording and plotting the training loss	4e (1), 4f (1)	`T7_PyTorch_Basics`, `T6_Matplotlib`

2. Marks per operation (award each independently)

Op	Marks	Marker looks for
1a	2	`read_csv` for orders (1) and for customers (1)
1b	1	`.shape` of BOTH printed
1c	3	detect: `duplicated().sum()` or equivalent (1); counts for both files printed: 3 and 4 (1); `drop_duplicates()` result **assigned back** (1)
1d	3	`merge(..., on="CustomerID")` (1); inner join chosen/stated (1); shape (124, 10) and "6 left out" printed (1)
1e	2	only the 5 columns selected (1); two conditions combined with `&` and brackets, 10 rows (1)
1f	1	`to_csv("merged_orders.csv", index=False)` ; file has 124 rows + header, 10 columns
2a-2d	2 each	right plot type of the right data (1); title (0.5) + both axis labels (0.5)
3a	3	`connect("ShopDB.db")` (1); DataFrame stored as table `Orders` (1); repeat-run safe + count 124 printed (1)
3b	2	SELECT with WHERE and ORDER BY ... DESC (1); `?` placeholders with a tuple of values (1)
3c	3	UPDATE with `?` (1); before AND after shown (1); `commit()` (1)
3d	2	`fetchall()` (or `read_sql_query`) retrieval (1); CSV exported with column headings (1)
4a	3	`torch.manual_seed(42)` BEFORE data (1); `torch.randn(200, 3)` (1); `y = X @ w + b + 0.1 * noise` with w of shape (3,1) (1)
4b	1	shapes printed: (200, 3) and (200, 1)
4c	2	class extends `nn.Module` with `super().__init__()` (1); Linear(3,8) -> ReLU -> Linear(8,1) in `forward` (1)
4d	1	model created, `MSELoss`, `SGD(lr=0.05)` all correct
4e	2	100-epoch loop with zero_grad, forward, loss, backward, step (1); loss recorded with `.item()` and printed at the required epochs (1)
4f	1	`plt.plot(losses)` with title and axis labels

Style marks (names, comments, layout) apply across the file. A crash in an early section loses the later sections - comment out the broken operation rather than submit a program that does not run.

3. Expected results (verified by running the solution twice, and by

independent plain-Python calculation with `csv`, `decimal`, `statistics` and, for the regression, a least-squares fit)

```
customers shape: (83, 5)         orders shape: (134, 6)
duplicate customer rows: 3       duplicate order rows: 4
after removal: (80, 5) (130, 6)
merged (inner join) shape: (124, 10)
orders with no matching customer: 6
North orders over 50: 10 rows    (first: O0003 Ella Dunn Games 82.42)

Rows stored in Orders: 124
North orders over 50 (biggest first): 10
   ('O0126', 'Hamish Evans', 'Games', 125.84)
   ('O0065', 'Aisha Hughes', 'Games', 122.7)
   ('O0072', 'Ella Dunn', 'Games', 106.3)
rows updated: 23
Books total BEFORE: 1011.17   AFTER: 965.17     (1011.17 - 23 x 2.00)
exported 23 rows to books_after_update.csv      (first row O0116 78.68)

X shape: (200, 3)  y shape: (200, 1)
Epoch   1/100  loss = 4.4789       Epoch  81/100  loss = 0.0516
Epoch  21/100  loss = 0.1824       Epoch 100/100  loss = 0.0445
Epoch  41/100  loss = 0.0947
Epoch  61/100  loss = 0.0650
```

The merged CSV has 124 rows and columns `OrderID, CustomerID, OrderDate, Category, Items, Amount, Name, Region, Age, LoyaltyYears`.

What your plots should show:

- **Scatter:** upward trend (correlation of Items and Amount is 0.48).
- **Line (monthly total, GBP):** Jan 569.50, Feb 721.28, Mar 476.49, Apr 274.34, May 220.36, Jun 520.12, Jul 208.30, Aug 316.49, Sep 553.56, Oct 535.54, Nov 436.46, Dec 468.81 (peak February, trough July).
- **Bar (total per category):** Games 2671.31, Music 1046.12, Books 1011.17, Stationery 572.65.
- **Box (amount per region, median / n):** East 34.60 / 30, North 23.52 / 41, South 24.07 / 35, West 49.38 / 18 (West highest, with the highest outlier near 175).
- **Loss curve:** starts near 4.5 (about the variance of y, 5.0) and falls steeply, then flattens near 0.04. It cannot go below the noise floor of 0.01 (0.1 squared). A least-squares fit finds weights 2.01, -1.00, 0.50 and bias 0.70, i.e. the "true" w and b, so the network is learning the right thing. Last decimals can differ by machine; the shape must not.

Idempotence: run the file a second time without deleting anything: every printed number above (including "BEFORE 1011.17 / AFTER 965.17") must be identical.

4. Planted traps in the data and how the solution handles them

Trap	Effect if missed	Handling
3 exact duplicate customer rows (customers who also have orders)	merge multiplies their orders: shape (134, 10) instead of (124, 10)	`drop_duplicates()` BEFORE merging
4 exact duplicate order rows	with customers also un-cleaned the merge is (138, 10)	same
6 orders for customers C101-C104 (not in `customers.csv`)	inner join drops them (124); left join keeps them as 130 rows with NaN Region/Name; outer join gives 152 rows	inner join stated in the paper; "6 left out" = `len(orders) - len(merged)`
22 customers who have never ordered	`customers.merge(orders, how="left")` gives 146 rows (NaN orders); only an outer/left-from-customers join shows them	direction of the join matters: orders first, inner
Region filter with `and`	`ValueError: truth value of a Series is ambiguous`	use `&` and put each condition in brackets
`df.drop_duplicates()` not assigned	duplicates still there (it returns a new table)	`df = df.drop_duplicates()`
`to_sql` run twice	default raises `ValueError: Table 'Orders' already exists.`; `if_exists="append"` doubles the table to 248 rows; and a persisted table would be discounted again on each run (965.17 -> 919.17 ...)	`if_exists="replace"` rebuilds from the clean DataFrame every run
`index=False` forgotten	`to_csv` / `to_sql` add an extra `index` column	`index=False`
UPDATE without `commit()`	change visible on this connection only, rolled back at close	`conn.commit()`
Single parameter `("Books")`	`ProgrammingError` (a string is not a 1-tuple)	`("Books",)`
Month labels	sorting dates as plain text still works for `YYYY-MM-DD`, but a line over `2024-01-05` style daily dates is a scribble	group by month first (`strftime("%Y-%m")`)
`plt.boxplot(labels=...)` or `tick_labels=...`	`labels` removed in matplotlib 3.11; `tick_labels` only exists in 3.9+	version-proof: `plt.boxplot(groups)` then `plt.xticks(range(1, n + 1), names)`
Seed set after generating data	different numbers every run	`torch.manual_seed(42)` first line of Section 4
`y` of shape `(200,)` with prediction `(200,1)`	`UserWarning: Using a target size ... different ...` and a silently wrong loss through broadcasting	make `w` shape (3,1) so `y` is (200,1)

5. Walkthrough of the individual operations

Pandas (analogy: two spreadsheets and a clipboard)

```
customers = pd.read_csv("customers.csv")      # file -> table in memory
print(customers.shape)                        # (rows, columns)
print(customers.duplicated().sum())           # how many photocopies
customers = customers.drop_duplicates()       # throw photocopies away
merged = orders.merge(customers, on="CustomerID", how="inner")
```

`duplicated()` marks every row that repeats an earlier row; `sum()` counts the `True`s. `merge` is a lookup: for each order, find the customer with the same `CustomerID` and glue the customer's columns on the right. **inner** = keep only matches (a guest list: no ticket, no entry); **left** = keep every order even when no customer matches.

```
sub = merged[(merged["Region"] == "North") & (merged["Amount"] > 50)]
sub = sub[["OrderID", "Name", "Region", "Category", "Amount"]]
merged.to_csv("merged_orders.csv", index=False)
```

The first line makes a True/False column and keeps the True rows (a sieve). The second picks columns (choosing which columns to look at).

Matplotlib (analogy: one sheet of paper per picture)

```
plt.scatter(merged["Items"], merged["Amount"])       # dots
plt.plot(monthly.index, monthly.values, marker="o")  # joined dots
plt.bar(per_cat.index, per_cat.values)               # bars
plt.boxplot(groups)                                  # list of 4 Series
plt.xticks(range(1, len(regions) + 1), regions)      # name the boxes
plt.title(...); plt.xlabel(...); plt.ylabel(...); plt.show()
```

A box plot summarises a pile of numbers: the box is the middle half, the orange line the median, the whiskers the usual range, circles the outliers. `groups` is a list with one Series of amounts per region. `groupby(months)` where `months` is a `"2024-01"`-style text per row makes one total per month, automatically in order.

SQLite (analogy: filing cabinet, clerk, "save" button)

```
merged.to_sql("Orders", conn, if_exists="replace", index=False)
cursor.execute("SELECT ... FROM Orders WHERE Region = ? AND Amount > ? "
               "ORDER BY Amount DESC", ("North", 50))
cursor.execute("UPDATE Orders SET Amount = ROUND(Amount - ?, 2) "
               "WHERE Category = ?", (2.0, "Books"))
conn.commit()
columns = [d[0] for d in cursor.description]
pd.DataFrame(cursor.fetchall(), columns=columns).to_csv("x.csv", index=False)
```

`to_sql` copies the whole table into the cabinet in one go. Each `?` is a blank on a form filled from the tuple, in order. `WHERE` is the filter; without it UPDATE changes EVERY row. `commit()` is "Save". `cursor.description` holds the column names so the export has headings (you may use `csv.writer` or `pd.read_sql_query(sql, conn, params=...)` instead).

PyTorch (analogy: a hiker finding the bottom of a valley)

```
torch.manual_seed(42)                         # same dice every run
X = torch.randn(200, 3)                       # 200 rows, 3 features
y = X @ w + 0.7 + 0.1 * torch.randn(200, 1)   # (200,3)@(3,1) -> (200,1)
```

`@` is matrix multiplication: each row of X is multiplied by the 3 weights and added up, giving one number per sample. Shapes must line up: inner sizes (3 and 3) cancel, leaving 200 x 1.

```
for epoch in range(100):
    optimizer.zero_grad(); loss = criterion(model(X), y)
    loss.backward(); optimizer.step(); losses.append(loss.item())
```

Wipe the whiteboard, guess, score, work out the slope, step downhill, write down the height (loss).

6. Viva-style follow-up questions

1. **What is the difference between an inner and a left join? Which gave you 124 and which 130 rows?** Inner keeps only orders with a customer match (124). Left keeps every order, filling the customer columns with NaN for the 6 unmatched ones (130).
2. **Why did you remove duplicates before merging, not after?** A duplicated customer row matches each of its orders twice, so the merge would repeat those orders (134 instead of 124). Cleaning afterwards would have to untangle that.
3. **What does `index=False` do?** It stops pandas writing the 0,1,2... row labels as an extra column in the CSV or table.
4. **Why use `?` placeholders instead of an f-string in SQL?** The values are passed separately from the SQL text, so quotes in the data cannot break the query and it is protected against SQL injection.
5. **What happens if you forget `WHERE` in the UPDATE?** Every row's amount would be reduced, not just books. (Count rows updated: 23, not 124.)
6. **Why does the final output layer have no activation here, unlike a classification network?** The target is any real number (it can be negative); a Sigmoid would squash predictions into 0..1.
7. **Why set the seed before generating data and creating the model?** Both the data and the starting weights are random; fixing the seed makes every run (and every marker) see identical numbers.
8. **Why does the loss never reach 0?** The data contains noise with standard deviation 0.1, so even the perfect model has MSE about 0.01. Going much lower would mean over-fitting the noise.
9. **How would you make the update safe to run twice?** The program rebuilds the table from the DataFrame each run (`if_exists="replace"`); otherwise store a flag/voucher column or only update rows where `Amount` has not been reduced yet.